Primeiros Passos com Django

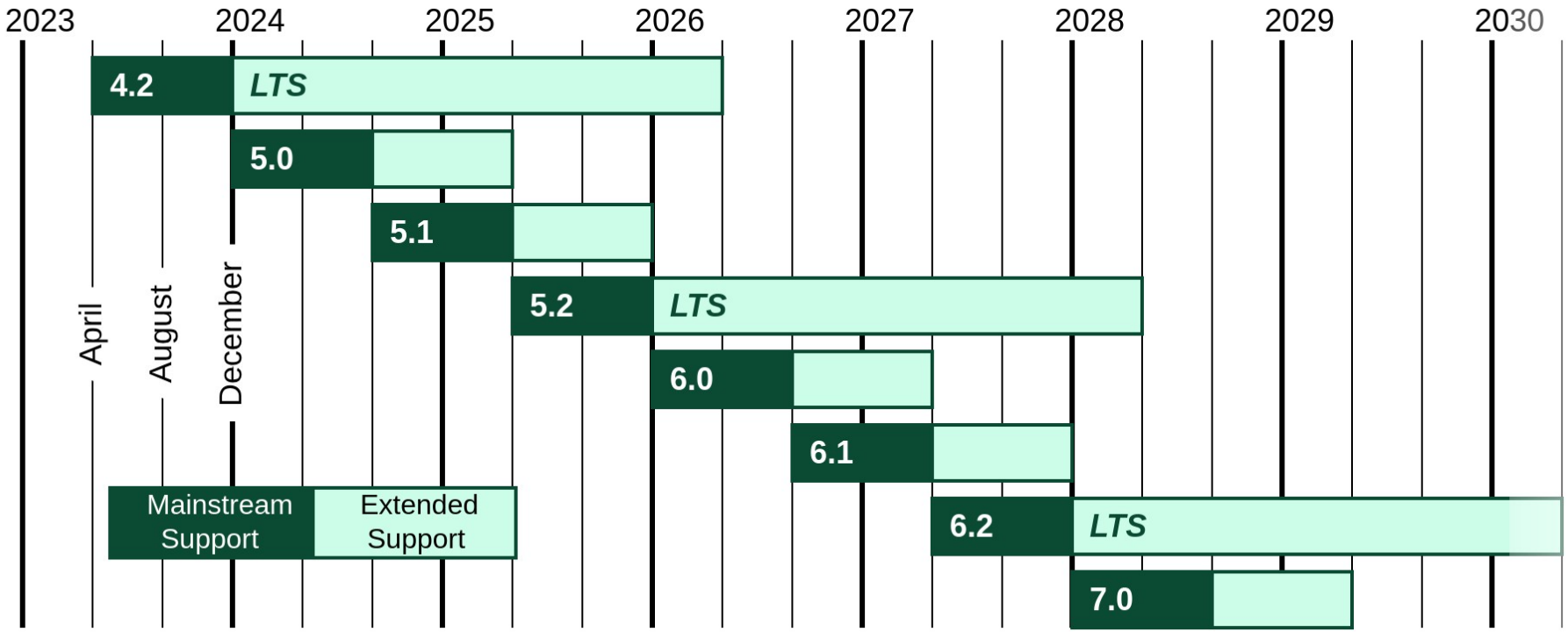
Prof. Dr. João Paulo Lemos Escola
Copyright© 2026

O que veremos?

- Introdução ao framework Django
- Instalação do ambiente
- Projeto básico

Django

- Framework web full stack
- Gratuito, OpenSource
- Desenvolvido em Python
- Lançado em 2005
- Atualmente na versão 6.0.2 (5.2 LTS)


2023
2024
2025
2026
2027
2028
2029
2030
April
August
December
4.2
LTS
5.0
5.1
5.2
LTS
6.0
6.1
6.2
LTS
7.0
Mainstream Support
Extended Support

https://www.djangoproject.com

django The web framework for perfectionists with deadlines.

OVERVIEW DOWNLOAD DOCUMENTATION NEWS COMMUNITY CODE ISSUES ABOUT ♥ DONATE

Django makes it easier to build better web apps more quickly and with less code.

Comece a usar Django

Meet Django

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

Download latest release: 5.1.6

DOCUMENTAÇÃO DO DJANGO ›

Support Django!

https://www.w3schools.com/django/

Tutorials Exercises Certificates Services Search...

HTML CSS JAVASCRIPT SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

Django Tutorial

- Django Home
- Django Intro
- Django Get Started
- Create Virtual Environment
- Install Django
- Django Create Project
- Django Create App
- Django Views
- Django URLs
- Django Templates
- Django Models
- Django Insert Data
- Django Update Data
- Django Delete Data
- Django Update Model

Learn more

Django Tutorial

❮ Home

Django is a back-end server side web framework.

Django is free, open source and written in Python.

Django makes it easier to build web pages using Python.

Start learning Django now ❯

MVT design pattern

- Model View Template
  - **M**odel: representações do mundo real (classes)
  - **V**iew: gerenciador de requisições do usuário
  - **T**emplate: interface do usuário (Frontend)
    (HTML + CSS + JS)

Model

- **models.py**
- Manipula os dados do banco
- ORM: Object Relacional Mapping
- Não é necessário escrever SQL

View

- **views.py**
- Métodos que executam requisições HTTP
- Contém a lógica de negócios
- Buscam dados dos models
- Enviam dados para os templates

Templates

- **Pasta** (diretório)
- Arquivos HTML que apresentam os resultados ao usuário
- HTML, CSS, JS + tags django:

```
<h1>My Homepage</h1>

<p>My name is {{ firstname }}.</p>
```

Variável de template

URLs

- **urls.py**
- Permite navegação entre as páginas
- Direciona o usuário para diferentes views

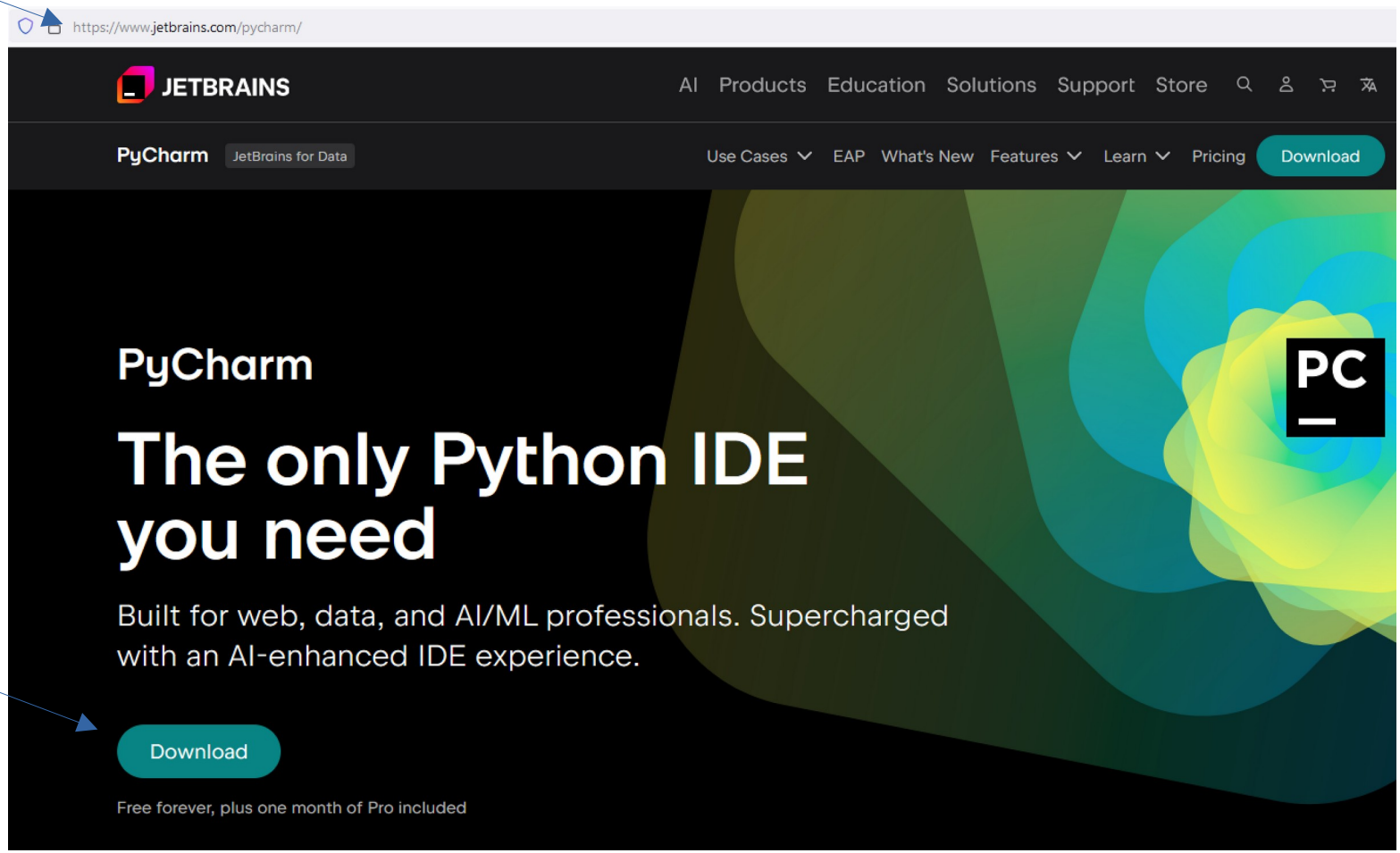
Como funciona?

- Instalar o Django e criar o primeiro app
- Abrir o navegador e chamar a URL:
  - Django recebe a requisição
  - **urls.py** direciona para **views.py**
  - Modelos são acionados em **models.py**
  - Dados são enviados para o **template**
  - O template (HTML) é entregue como resposta

PyCharm

- IDE gratuita OpenSource para Python;
- Mantida pela empresa JetBrains;
- **Community** Edition (descontinuada).

Baixando e instalando



https://www.jetbrains.com/pycharm/download/?section=windows

JETBRAINS

AI Products Education Solutions Support Store

PyCharm JetBrains for Data

Use Cases EAP What's New Features Learn Pricing Download

Windows macOS Linux

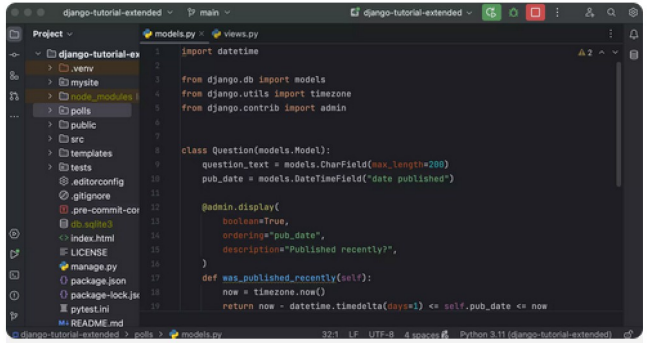
PyCharm

Unified Product

The only Python IDE you need

Download

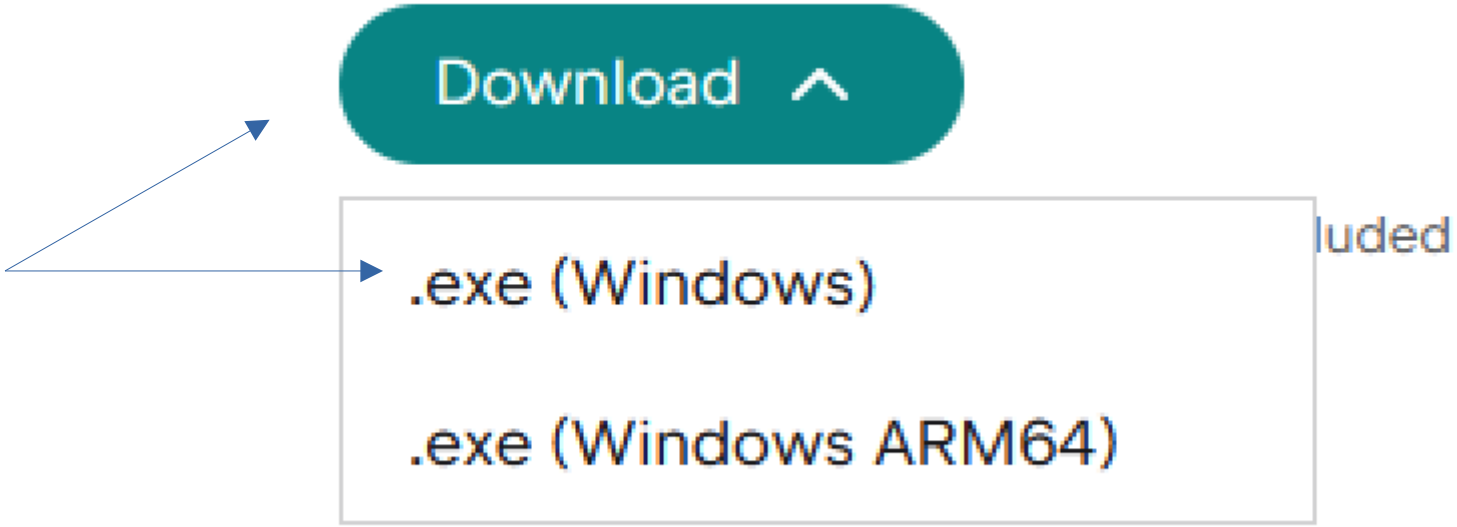
Free forever, plus one month of Pro included



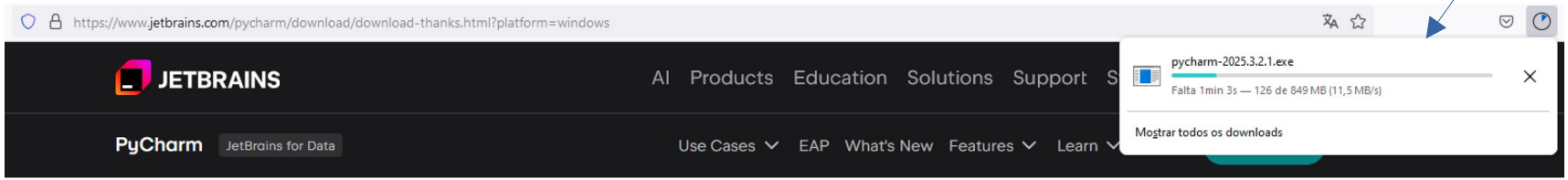
Version: 2025.3.2.1
Build: 253.30387.173
1 February 2026

System requirements
Installation instructions
Other versions
Third-party software

Baixar a versão Windows



Aguarde o download



Thank you for downloading PyCharm!

Your download should start shortly. If it doesn't, please use the direct link.

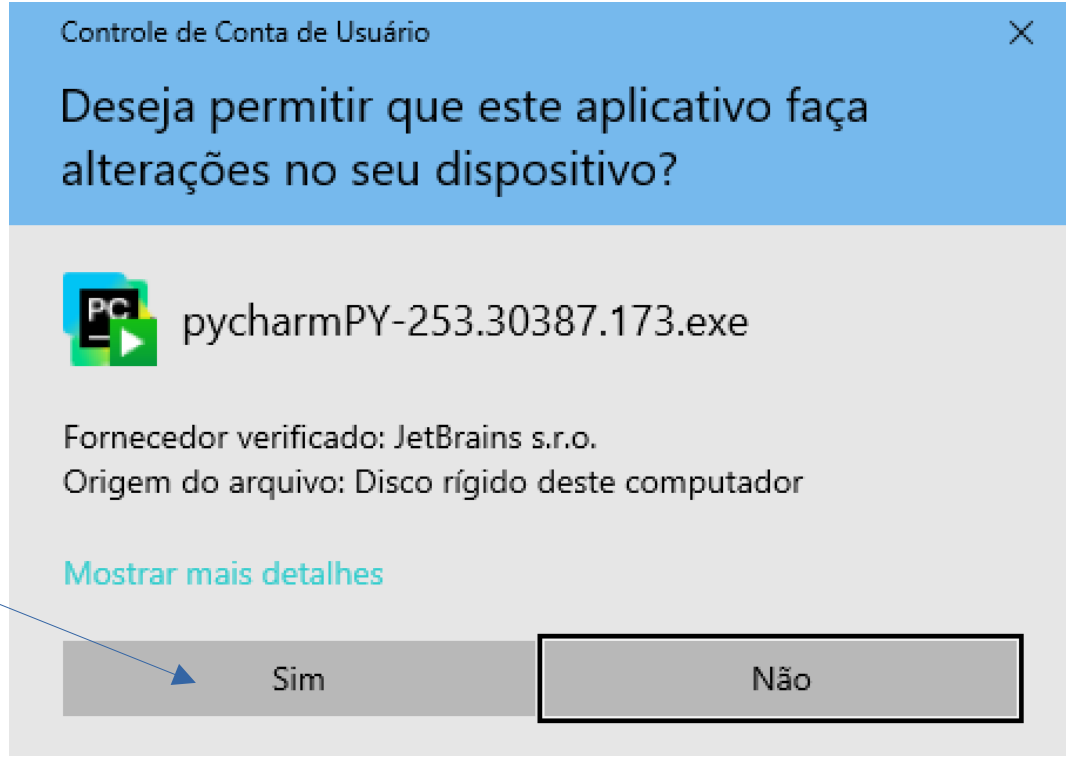
Download and verify the file SHA-256 checksum.

Learn more about the digital signatures of JetBrains binaries.

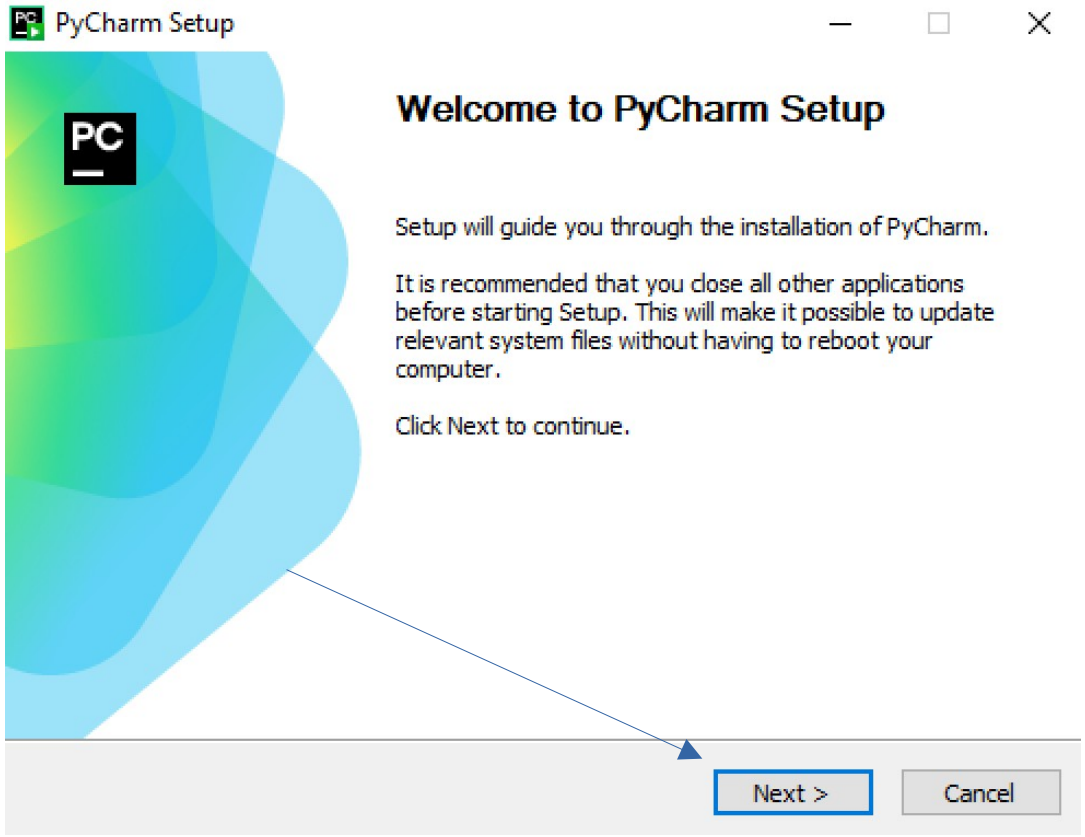
PyCharm goes AI! Less routine, more coding joy. All refined JetBrains AI tools – right in your IDE, for free.

Get started

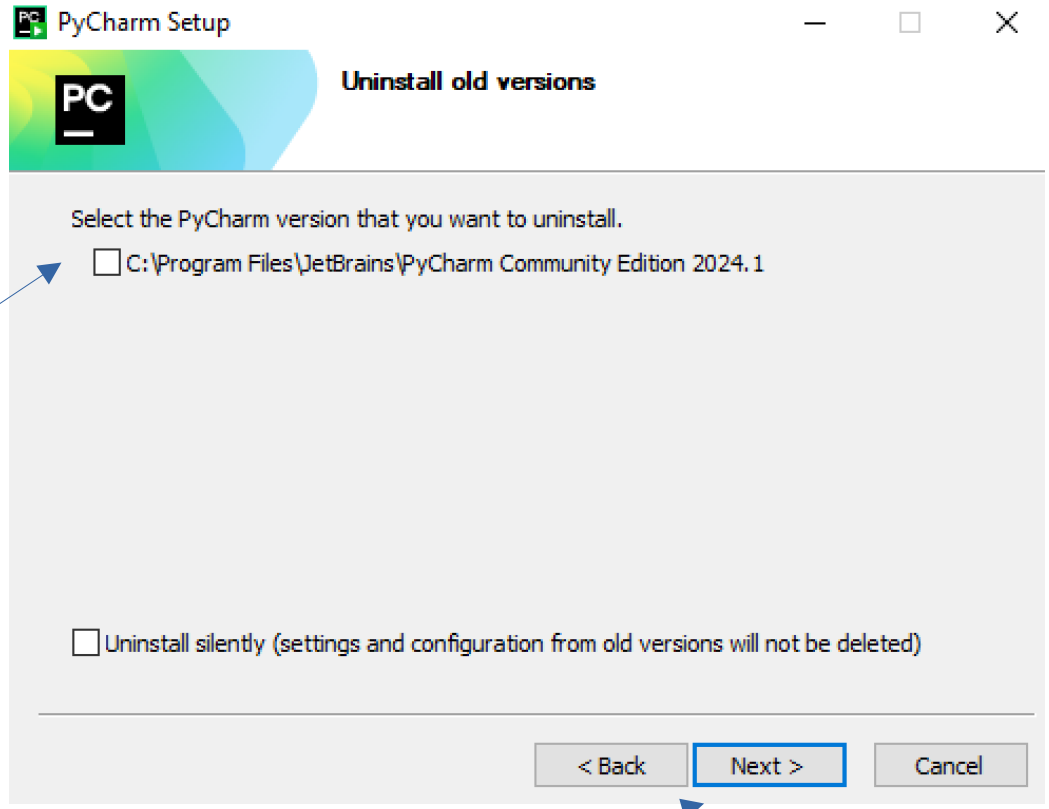
Execute o arquivo baixado



Iniciando a instalação

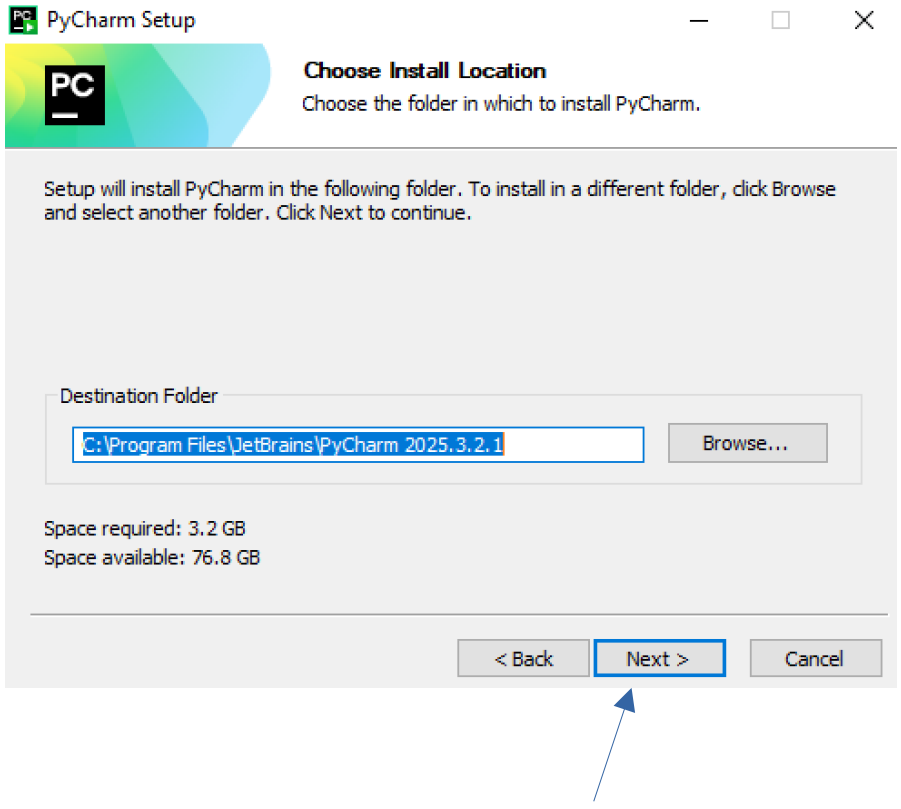


Desinstalando versão antiga



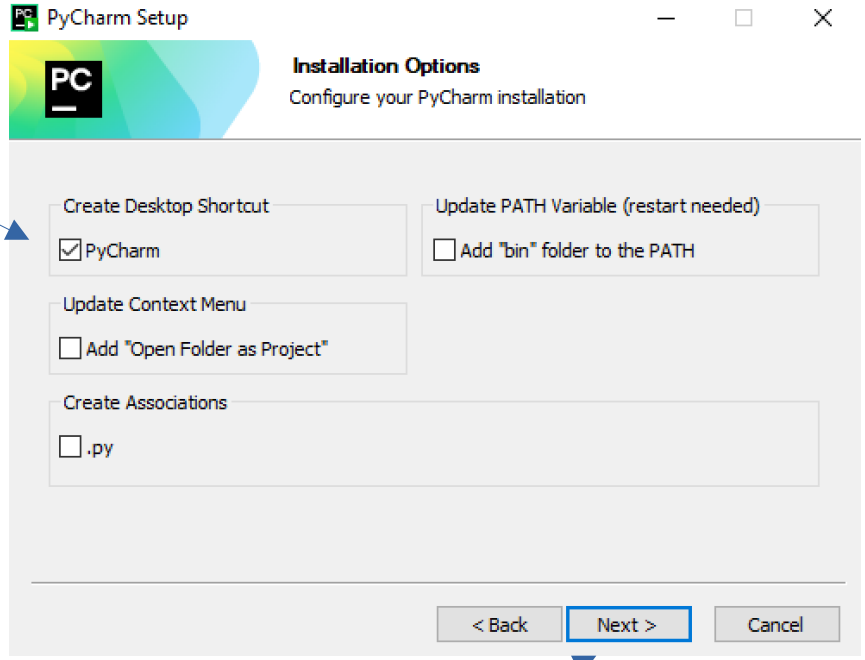
Diretório de instalação

- Deixe como está.



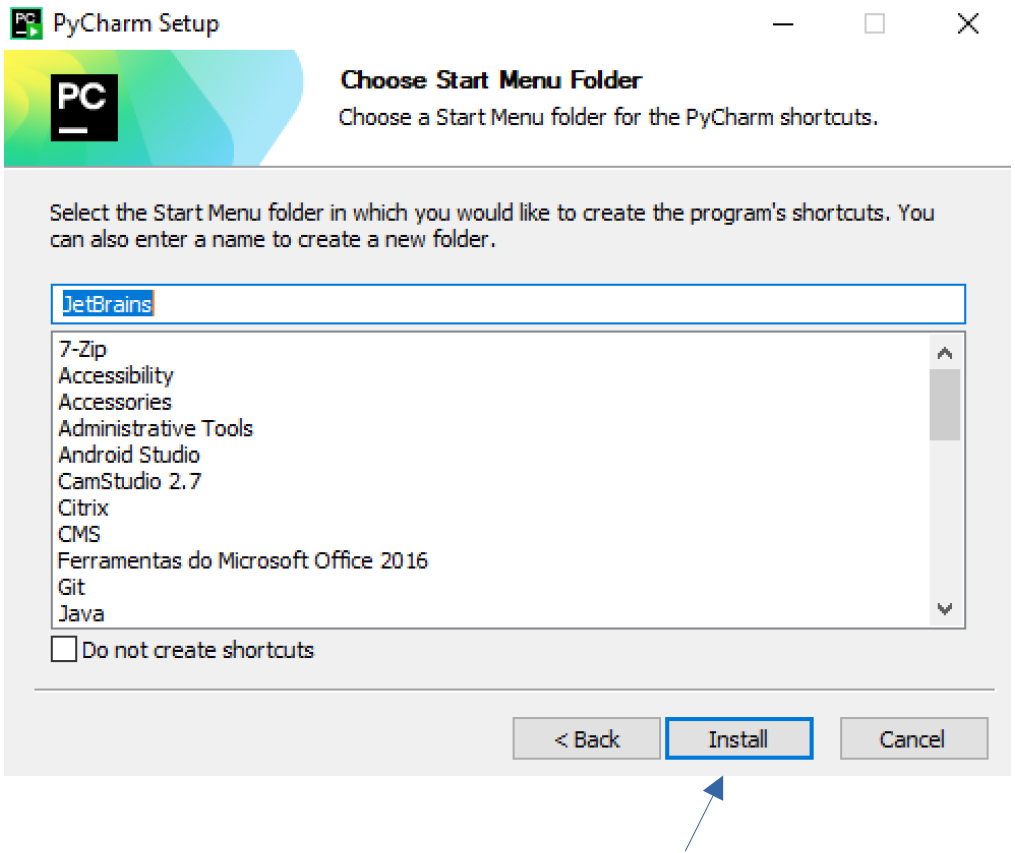
Opções da instalação

- Criar atalho na área de trabalho.

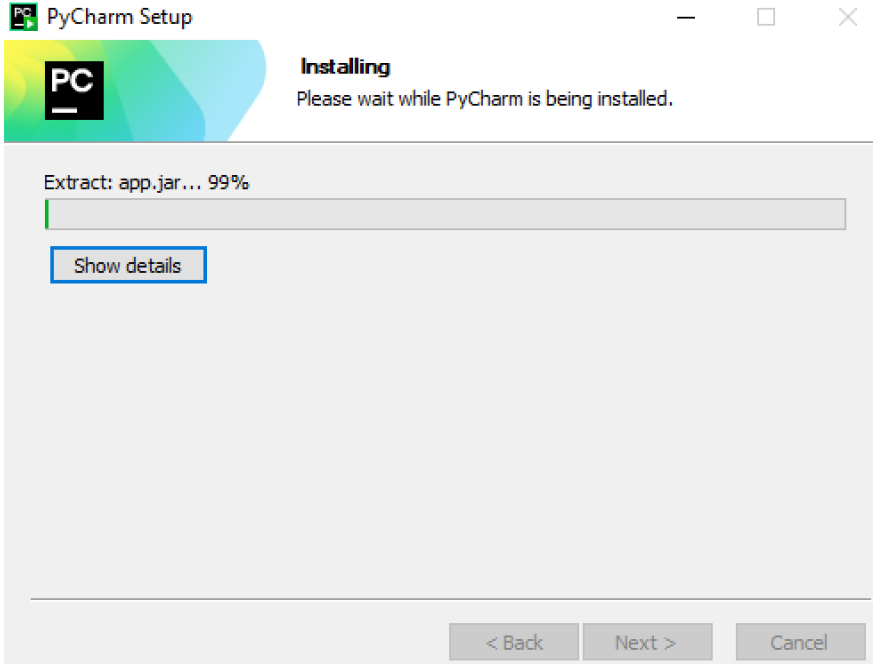


Iniciar a instalação

- Clique em Install.

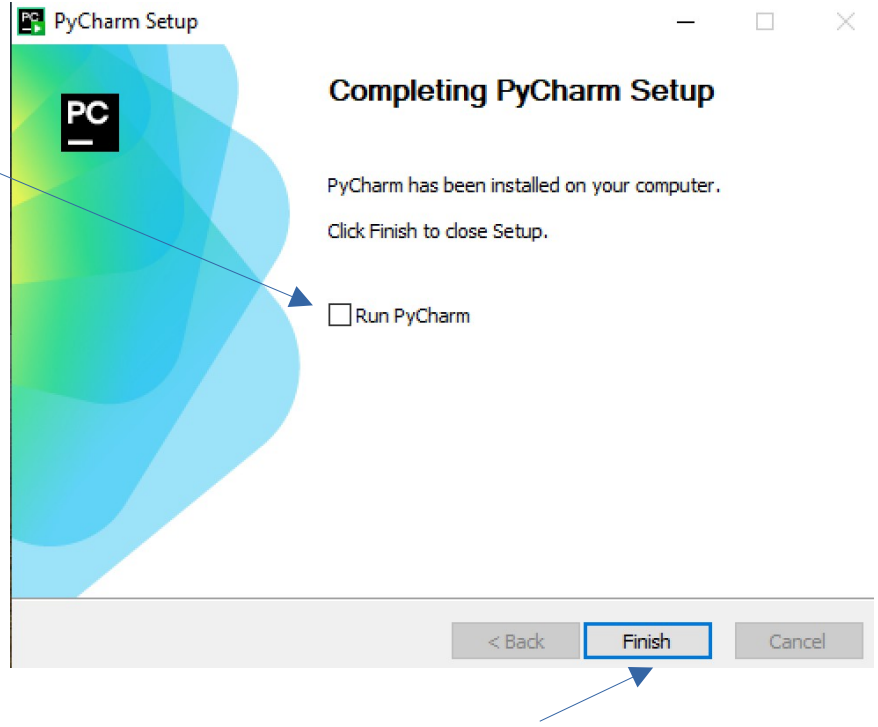


Aguarde a instalação



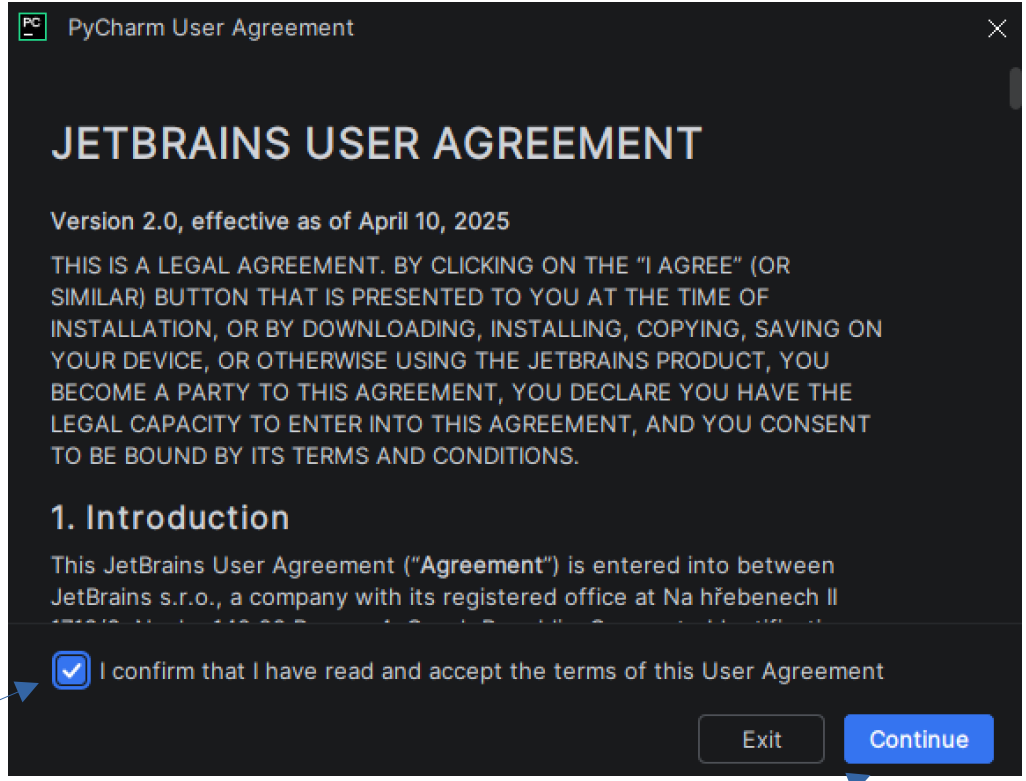
Instalação terminada

- Clique para iniciar a IDE agora ;
- Ou dê um duplo clique no atalho da área de trabalho.

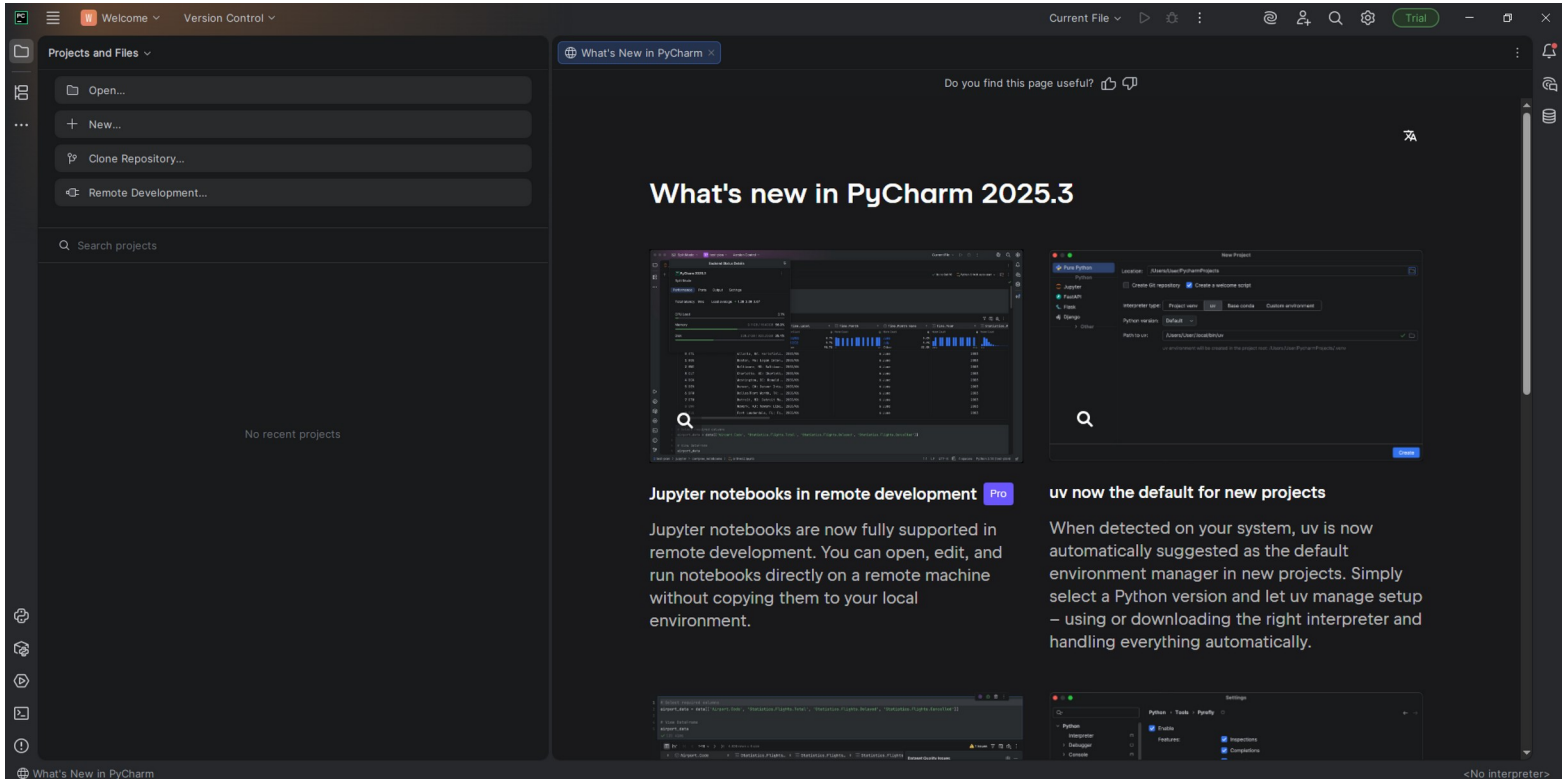


Termos de uso

- Vai aparecer somente na primeira vez.

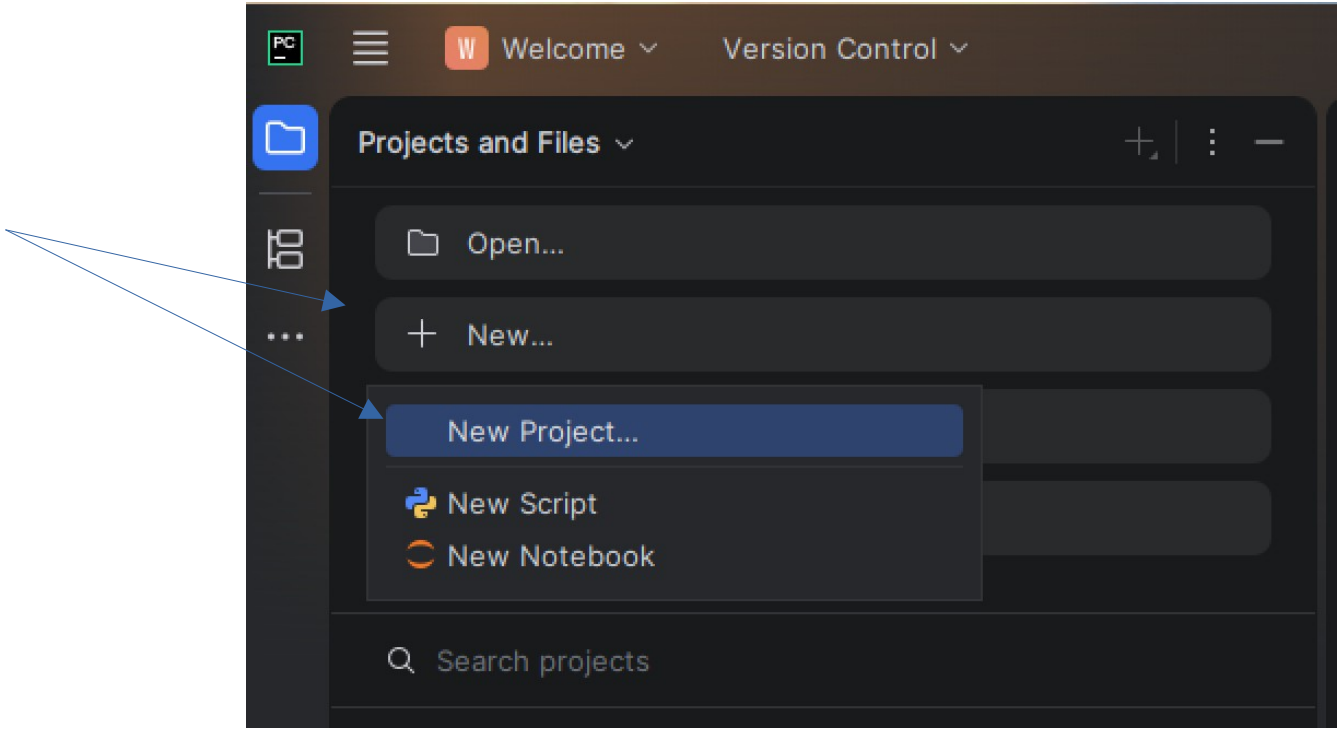


IDE está em execução



Criando um novo projeto

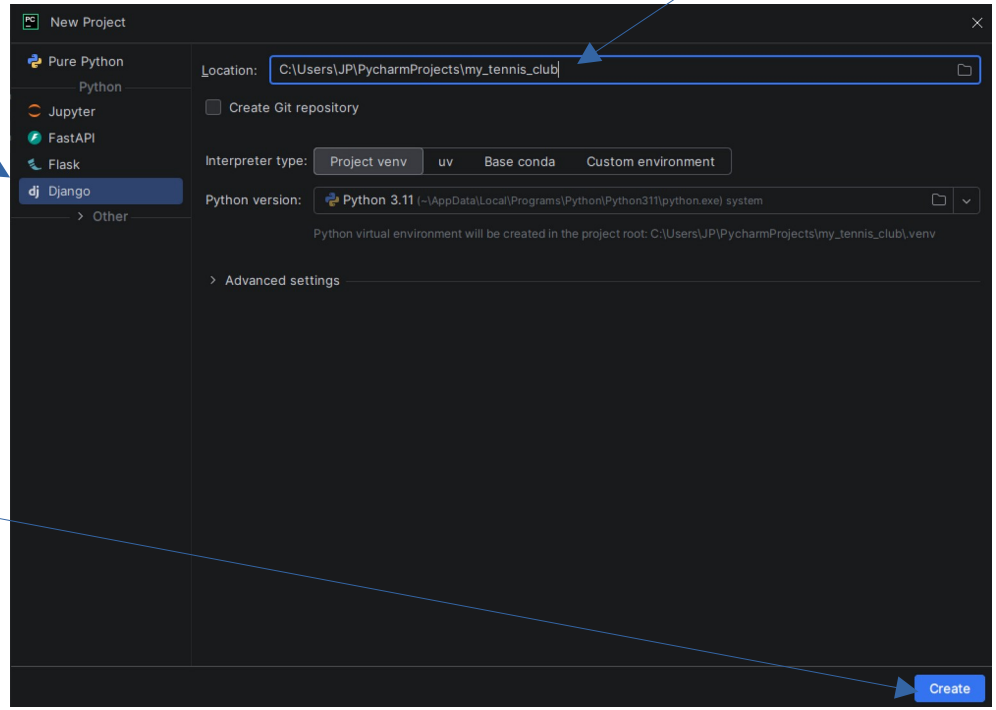
- Clique em New > New Project.



Tipo do projeto

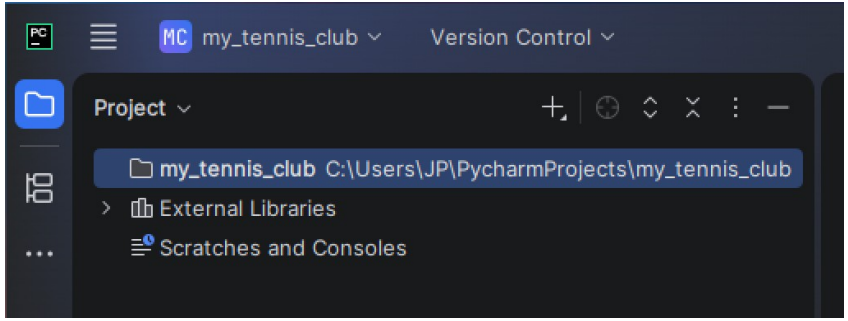
- Escolha Django;
- Defina o nome do projeto;
- Clique em Create.

Defina o nome do projeto;



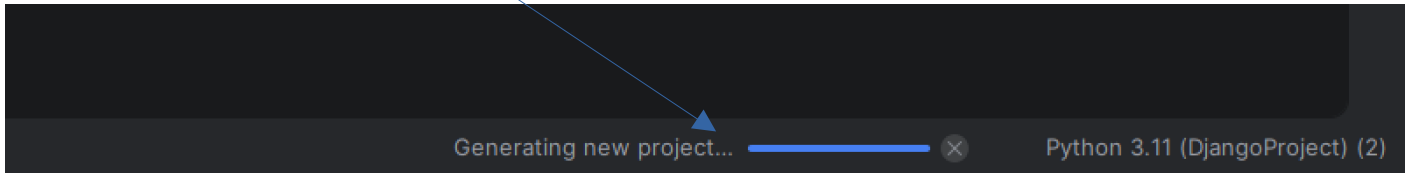
Aguarde a criação do projeto

- O projeto ainda está incompleto.



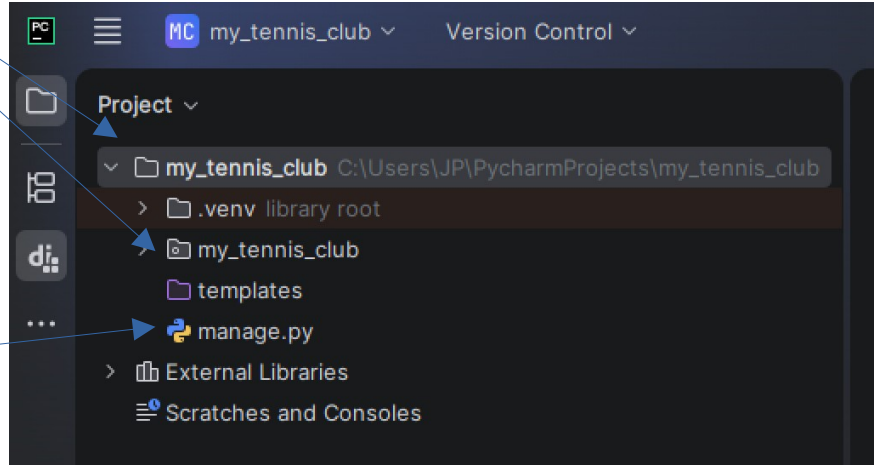
A barra no canto inferior direito mostra que estão sendo baixados os pacotes do projeto;

- Aguarde a barra sumir.



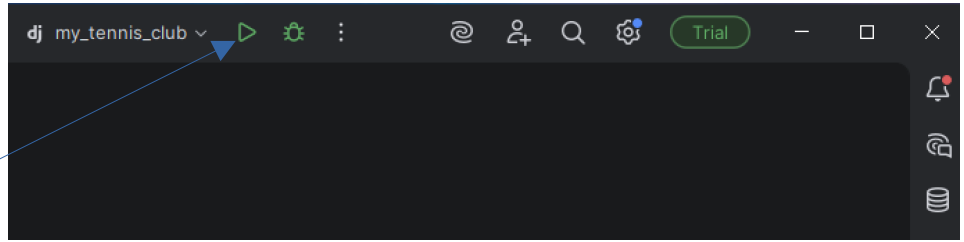
Projeto pronto!

- Veja que agora há uma pasta com o mesmo nome do projeto;
- Também foi incluído o arquivo **manage.py**, que é o arquivo de configuração do projeto.

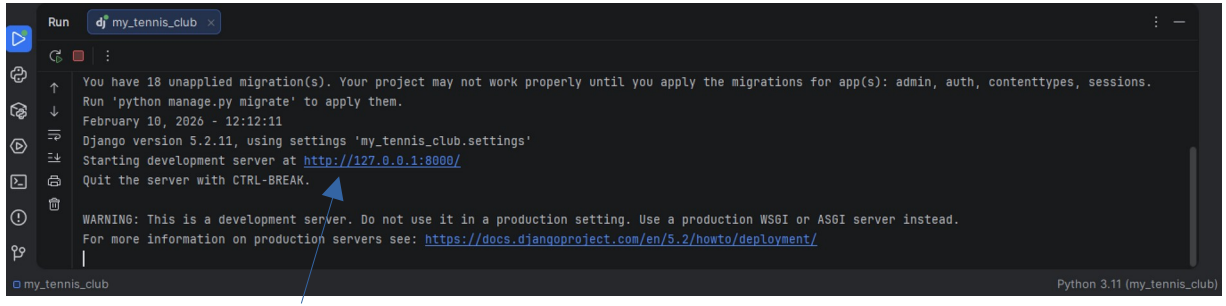


Executando o projeto

- Clique no **play** na barra superior;



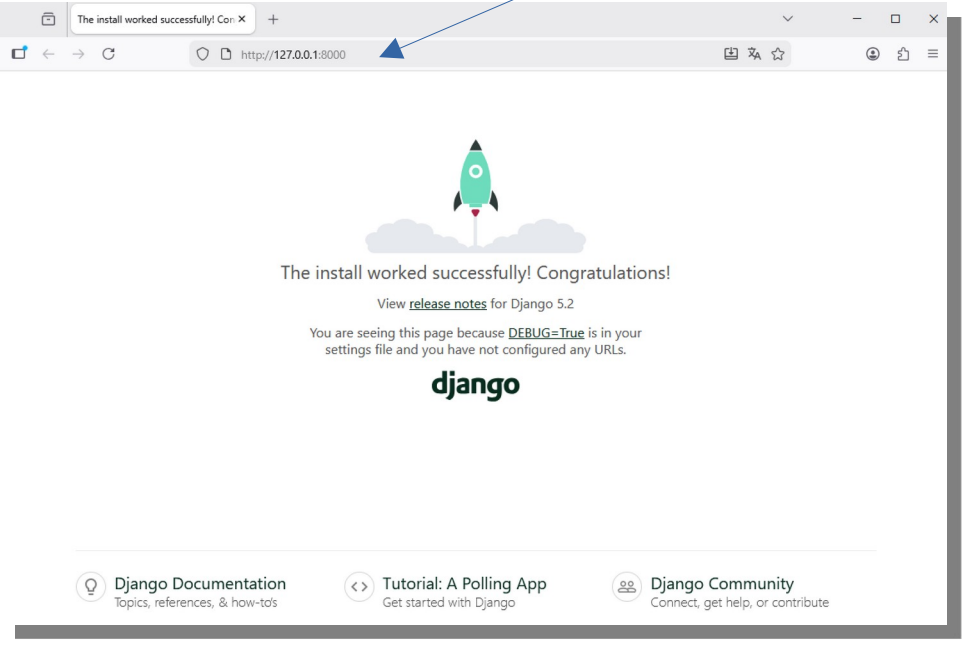
- Veja que o utilitário surge na barra inferior da IDE;



Clique no link para visualizar o resultado.

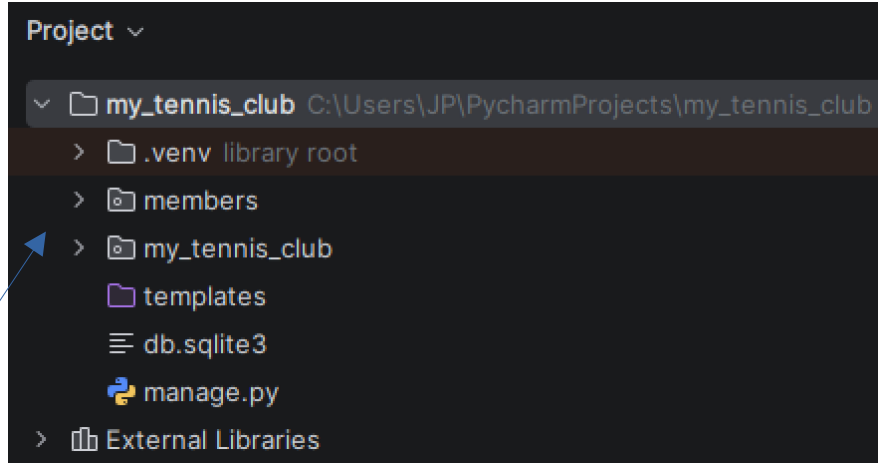
Projeto executado com sucesso!

- Por padrão, o servidor é executado no computador local (localhost), porta 8000.



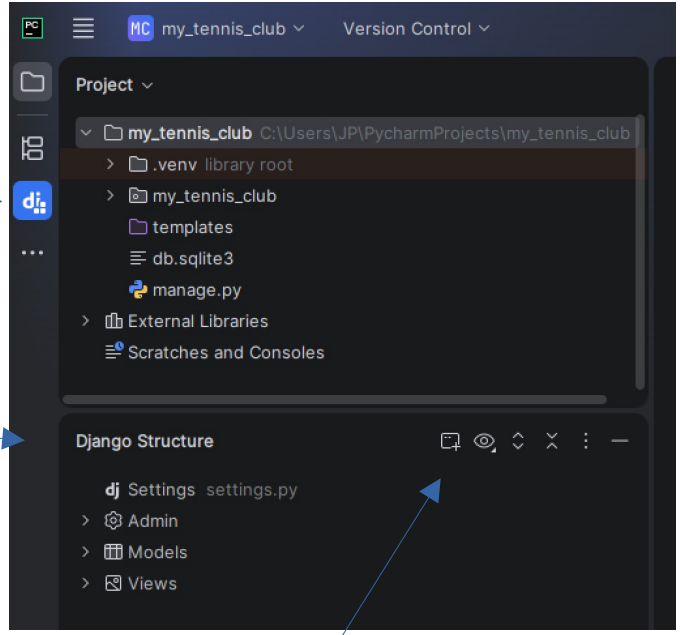
O que é um app?

- Um projeto pode conter múltiplas aplicações;
- Cada aplicação pode ter uma responsabilidade;
- Posteriormente podemos desativar/ativar apps;
- Vamos criar um app ‘members’ que nos permitirá gerenciar os membros do clube de tênis;
- Ao seguirmos os passos descritos a seguir, será criada uma **pasta para o app**, essa pasta também é chamada de módulo;



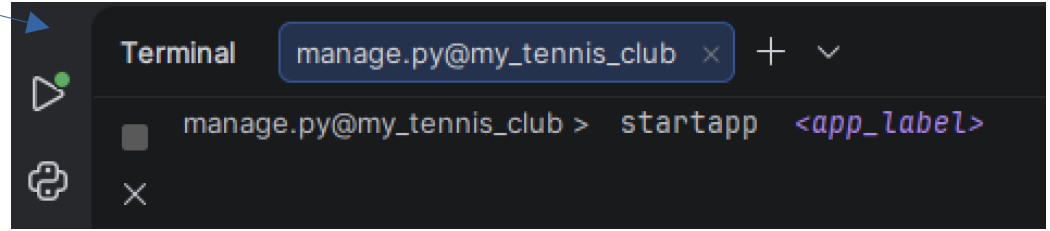
Criando um app

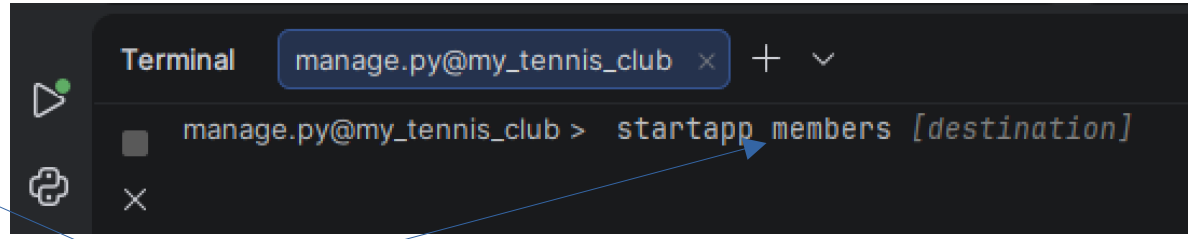
- Clique para habilitar o bloco de estrutura do projeto;
- O bloco vai surgir logo abaixo;
- Clique em **add** para definir o nome do app.



Comando para criar um app

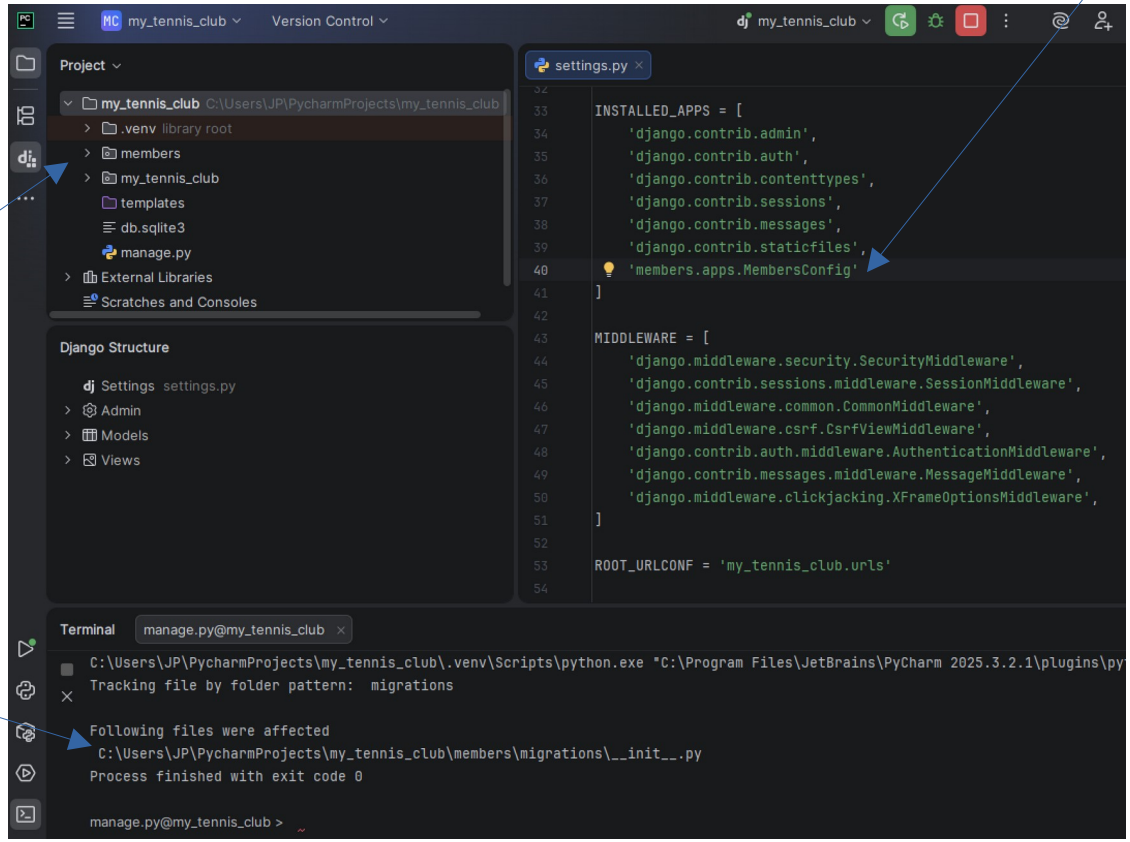
- Veja que o **terminal** surgiu com o comando ‘startapp’ para criar um app:
- O cursor já está posicionado aguardando a digitação do nome do app;
- Digite **members** e pressione Enter.





Ativando o app ‘members’

- Veja que o arquivo **settings.py** do projeto foi aberto e a linha 40 foi adicionada;
- Veja que o **módulo** python também foi criado;
- Confirmação de que o comando foi executado com sucesso.

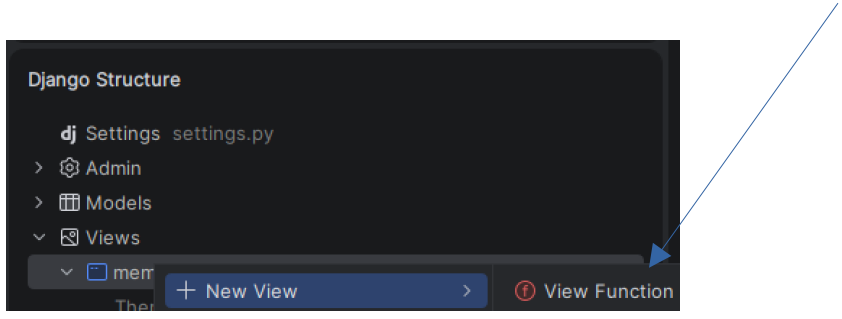


Programando

- O que já fizemos:
  - Criamos um projeto;
  - Testamos o projeto no servidor local;
  - Criamos um app.
- Tudo isso fizemos por meio de botões da IDE (também possível por meio de comandos Python) através arquivo manage.py;
- Agora vamos **começar a programar** nosso App usando a IDE PyCharm.

Definindo a view ‘members’

- Clique em Views;
- Clique com o botão direito em ‘members’ > new View > View Function;



Adicionando nossa view

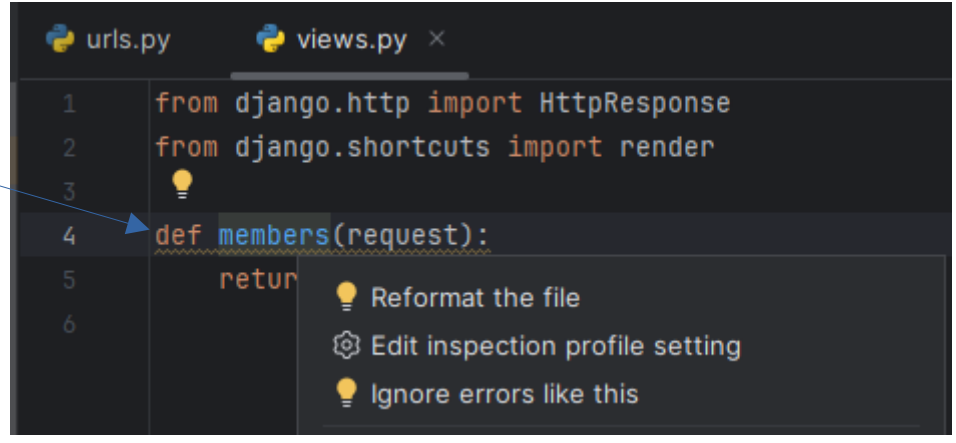
- No arquivo **views.py**:
  - Veja que a função foi criada;
  - Substitua-a pela função abaixo:

```python
from django.shortcuts import render

# Create your views here.
def view_name(request):
    pass
```

```python
from django.http import HttpResponse
from django.shortcuts import render

def members(request):
    return HttpResponse("Hello World!")
```

Código limpo

- Veja que a linha 4 está destacada;
- Clique na lâmpada (ou Alt+Enter) e escolha **Reformat the file**.



Definindo a url da view ‘members’

- **Crie** um arquivo **urls.py**
  - Clique na pasta ‘members’ > new > file

my_tennis_club/members/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('members/', views.members, name='members'),
]
```

urls.py do projeto

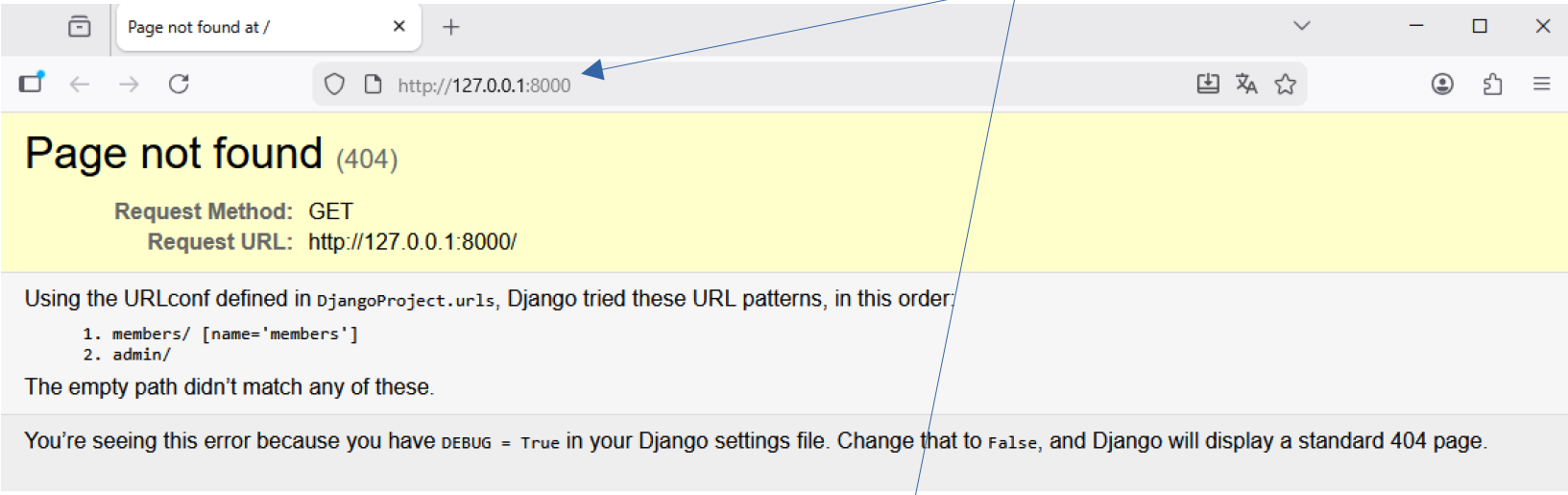
- Será necessário ajustar o arquivo **urls.py** do projeto:

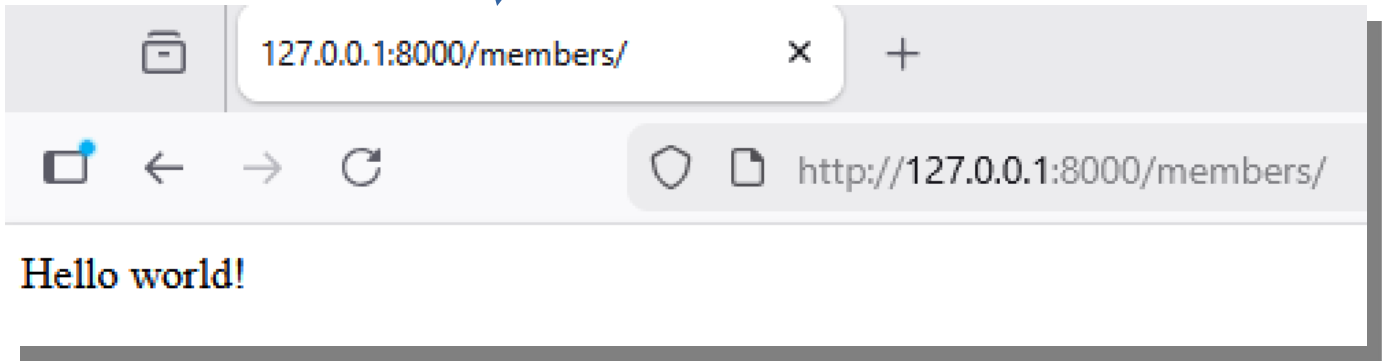
Dentro da pasta my_tennis_club

my_tennis_club/my_tennis_club/urls.py:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('', include('members.urls')),
    path('admin/', admin.site.urls),
]
```

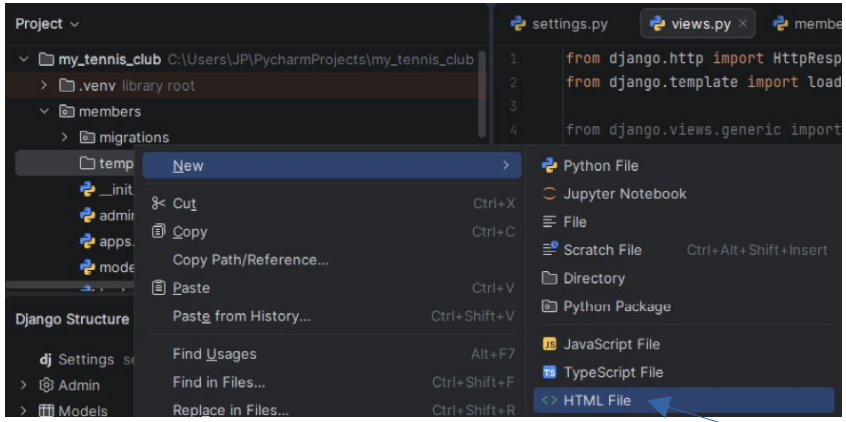
Veja o resultado

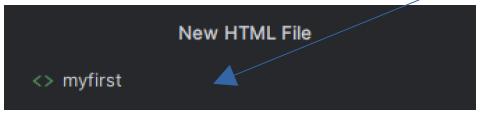




Definindo um template

- Crie a pasta ‘templates’ dentro de ‘members’
- Crie o arquivo ‘myfirst.html’





my_tennis_club/members/templates/myfirst.html:

```
<!DOCTYPE html>
<html>
<body>

<h1>Hello World!</h1>
<p>Welcome to my first Django project!</p>

</body>
</html>
```

Alterando a view

my_tennis_club/members/views.py:

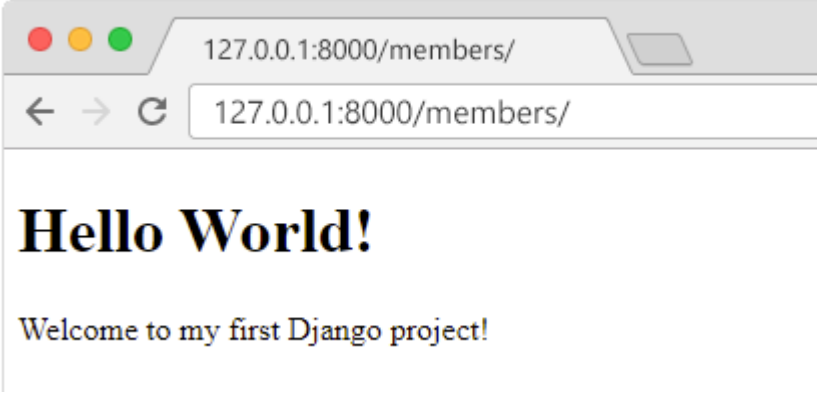
```
from django.shortcuts import render
from django.http import HttpResponse

def members(request):
    return HttpResponse("Hello world!")
```

my_tennis_club/members/views.py:

```
from django.http import HttpResponse
from django.template import loader

def members(request):
  template = loader.get_template('myfirst.html')
  return HttpResponse(template.render())
```

Testando o resultado

- Ao salvar um arquivo, o servidor é automaticamente atualizado com o novo código:

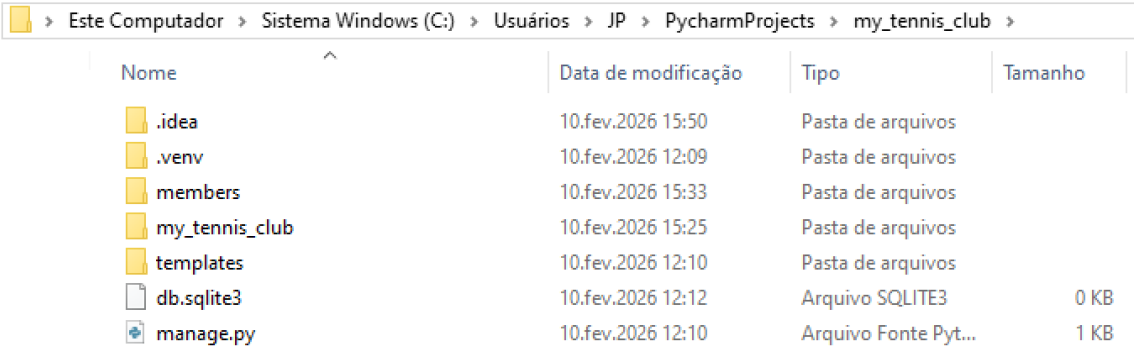


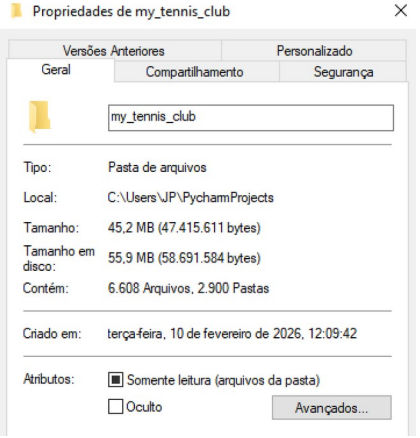
Forçando atualização do servidor

- Caso o servidor não atualize sozinho, é possível forçar pelo botão **reload** da barra superior da IDE:

Salvando o projeto

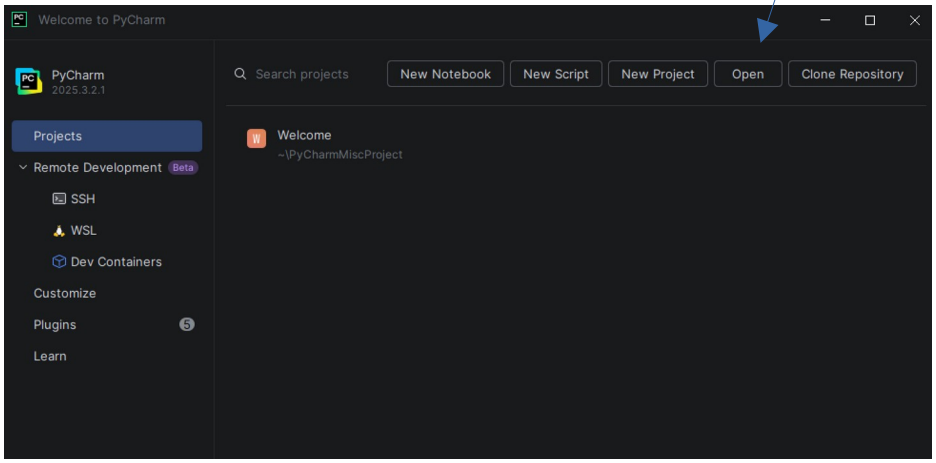
- Os projetos são salvos em c:\users\aluno\PycharmProjects:
  - Guarde a pasta do projeto em seu ‘Google Drive’ ou pendrive antes de desligar o computador;
  - Caso o projeto esteja muito grande, exclua as pasta **.venv**

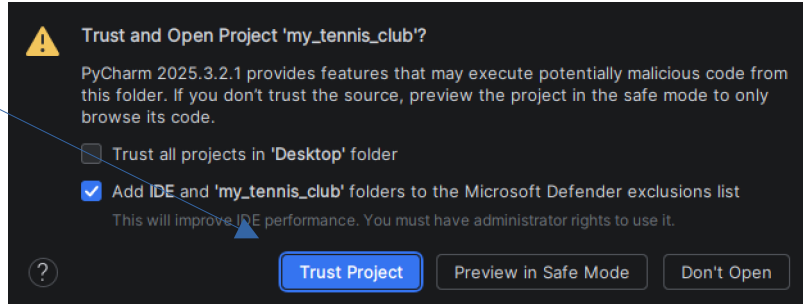




Abrindo um projeto

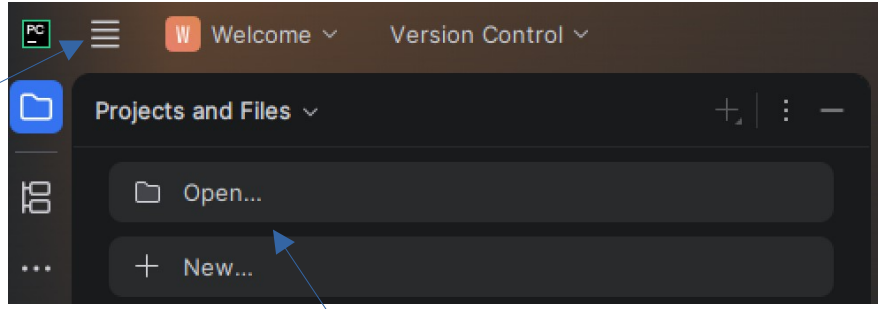
- Baixe seu projeto do seu drive virtual ou pendrive;
- Abra o PyCharm e clique em Open;
- Selecione a pasta do projeto e confirme;
- Aguarde a sincronização dos pacotes antes de executar ou alterar o projeto.

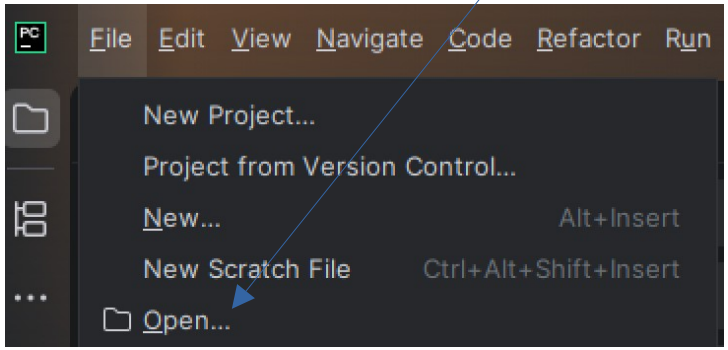




Outras formas de abrir

- Também é possível abrir um projeto pelo menu principal e depois clicando em Open;





Ou pelo botão “Open…” da tela principal, caso nenhum projeto esteja ativo no momento.

Problemas com .venv

- Caso não consiga abrir um projeto antigo por conta de conflitos no ambiente virtual:
  - Crie um novo projeto;
  - Cole os arquivos e pastas do projeto antigo;
  - Execute o projeto sem problemas.

O que vimos?

- Introdução ao framework Django
- Instalação do ambiente
- Projeto básico

Referências

- w3schools.com/django